

Onboarding contributeur

Construction PA

Février 2026



Parcours d'onboarding : `./docs/00_onboarding`



• Tronc commun

- 1. Contexte et vision du projet
- 2. Architecture : les 10 briques
- 3. Les normes de référence
- 4. Organisation du dépôt
- 5. Stratégie BDD

• Finalisation commune

- 10. Rapports de tests
- 11. Contribuer

• Parcours DEV

- 7. Installation environnement
- 8. Le code : packages et services
- 9. Implémenter un test BDD

• Parcours METIER

- 6. Rédiger un scénario BDD
- 12. Focus par brique

Renvoi vers une documentation du repo :
`./<rep>/<document>`

[TOUS] Contexte et vision du projet

Bloc 1 - Comprendre pourquoi ce projet existe

[TOUS] La reforme facturation électronique

- **Qu'est-ce qu'une PA – Plateforme Agréée (anciennement PDP - Plateforme de Dématérialisation Partenaire) ?**
 - Point d'entrée obligatoire pour émettre/recevoir des factures électroniques
 - Positionnement vis-à-vis du PPF (Portail Public de Facturation)
- **Calendrier de la reforme**
 - Sept. 2026 : obligation de réception pour toutes les entreprises
 - 2027 : obligation d'émission (échelonné par taille)
- **3 obligations : réception, émission, e-reporting (transmission fiscale)**

Vision PA Communautaire

- **Trajectoire du projet PDP Libre**

- Choix d'une PA partenaire pour septembre 2026
- En parallèle : construction d'une PA Communautaire open source

- **Valeurs fondamentales**

- Open source : code ouvert, licences libres (GPL-3.0-or-later)
- Souveraineté : indépendance vis-à-vis des acteurs commerciaux
- Communautaire : gouvernance participative et transparente
- Sans but lucratif : au service de l'intérêt général

- **Ecosystème : émetteur, destinataire, PA émettrice/réceptrice, PPF, DGFIP**

- Ce projet **open source** est porté par des bénévoles avec comme objectif d'accompagnement pour les contributeurs, l'acquisition de compétences dans la facturation électronique
- **L'objectif cible est produire tout ou partie du code d'une Plateforme Agrée candidat à être exploité par l'association PDF Libre ou toute entité qui le déciderait**

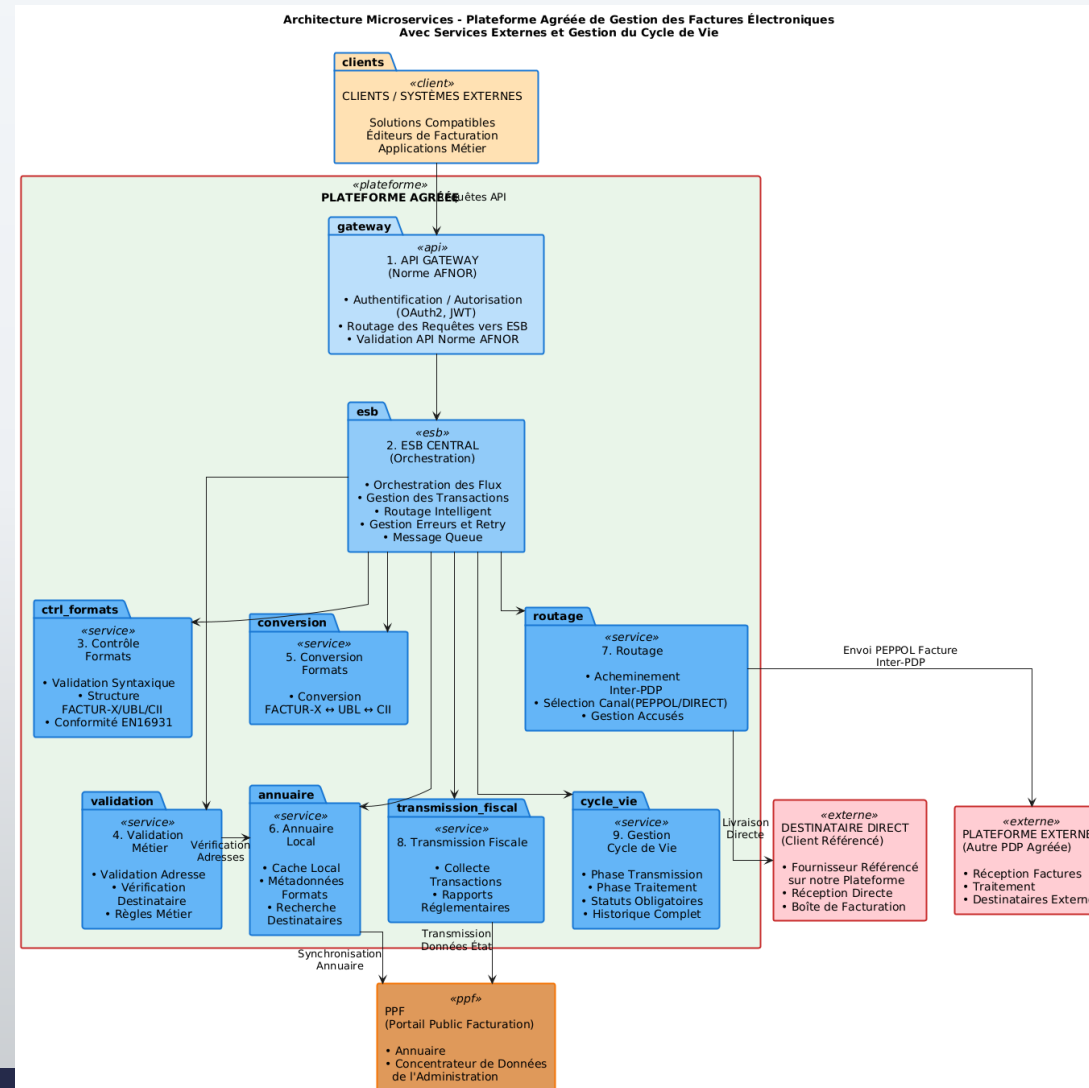
Parcours d'une facture (vue simplifiée)

- 1. L'émetteur dépose une facture sur sa PDP
- 2. La PDP contrôle le format (UBL, CII, Factur-X)
- 3. La PDP valide les règles métier
- 4. La PDP route la facture vers la PA du destinataire (via PEPPOL dans certains cas)
- 5. Le destinataire reçoit la facture et gère son cycle de vie
- 6. Les données fiscales sont transmises à l'Etat (e-reporting)
- Ressources : [./README.md](#), forum <https://forum.pdplibre.org/>

[TOUS] Architecture fonctionnelle

Bloc 2 - Les 10 briques de la plateforme

Vue d'ensemble de l'architecture



[TOUS] Les 10 briques



- **01 - API Gateway (FastAPI)**
 - Point d'entrée REST, norme XP Z12-013
- **02 - ESB Central (NATS)**
 - Orchestration des flux, message queue
- **03 - Contrôle Formats**
 - Validation UBL / CII / Factur-X
- **04 - Validation Métier**
 - Règles métier, vérification destinataire
- **05 - Conversion Formats**
 - Conversion entre UBL, CII, Factur-X
- **06 - Annuaire Local**
 - Cache local, synchronisation PPF
- **07 - Routage (PEPPOL)**
 - Acheminement inter-PDP
- **08 - Transmission Fiscale**
 - E-reporting vers l'Etat
- **09 - Gestion Cycle de Vie**
 - Statuts : déposée, reçue, approuvée...
- **10 - Stockage (S3 / SeaweedFS)**
 - Persistance des fichiers factures

Flux type d'une facture

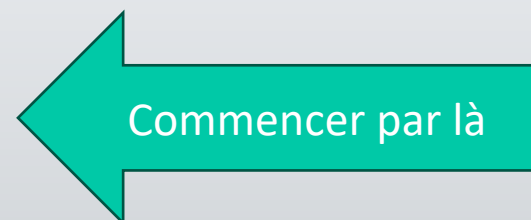
- **Dépôt facture via API Gateway (01)**
 - Publication sur l'ESB Central (02)
- **Contrôle du format (03) puis validation métier (04)**
- **Conversion si nécessaire (05)**
- **Consultation de l'annuaire (06) pour trouver le destinataire**
- **Routage vers la PA destinataire via PEPPOL (07)**
- **Transmission fiscale a l'Etat (08)**
- **Gestion du cycle de vie et statuts (09)**
- **Stockage des fichiers en S3 (10)**
- **Documentation par brique : `./docs/briques/<NN-nom>/README.md`**

[TOUS] Les normes de référence

Bloc 3 - AFNOR, UN/CEFACT et cas d'usage

3 normes AFNOR incluses dans le repo

- **XP Z12-012 - Formats et cycle de vie**
 - Formats : UBL, CII, Factur-X (EN16931)
 - Profils de messages, règles métier, statuts de cycle de vie
 - Annexe A (Excel) : règles métier, code lists, mappings
 - Annexe B : exemples de factures et statuts
 - **XP Z12-013 - API REST**
 - Swagger en annexe (Flow Service, Directory Service)
 - Routes : /healthcheck, /flows, /directory-line, /search...
 - **XP Z12-014 - Cas d'usage B2B**
 - UC1 a UC5 avec exemples XML complets
 - Scenarios nominaux et d'erreur
-
- **`./docs/norme/index.md`**
 - Fichiers PDF, XSD et exemples XML dans **`./docs/norme/`**



XP Z12-014 - Cas d'usage B2B

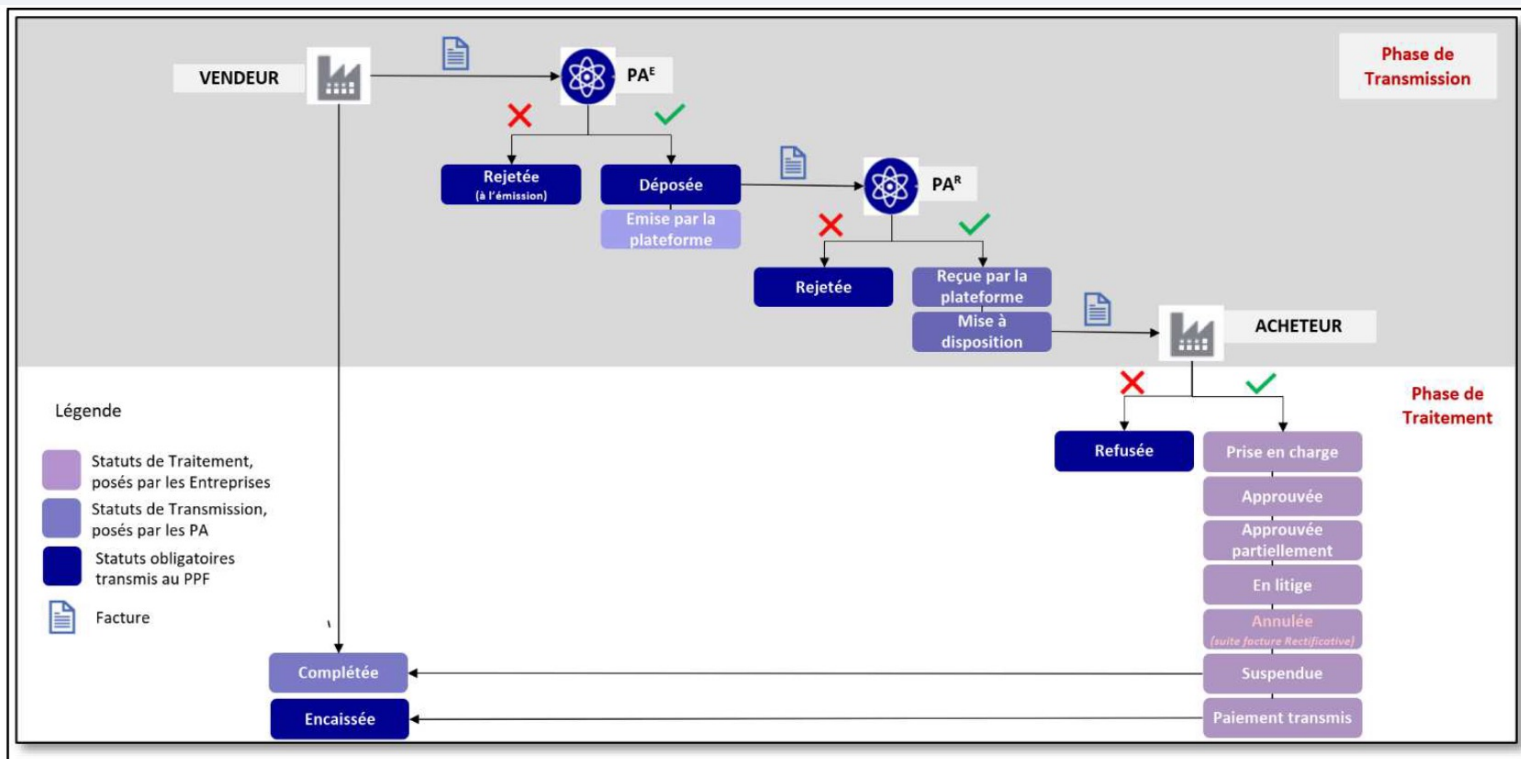


Figure 1 : Illustration du cycle de vie et des différents statuts

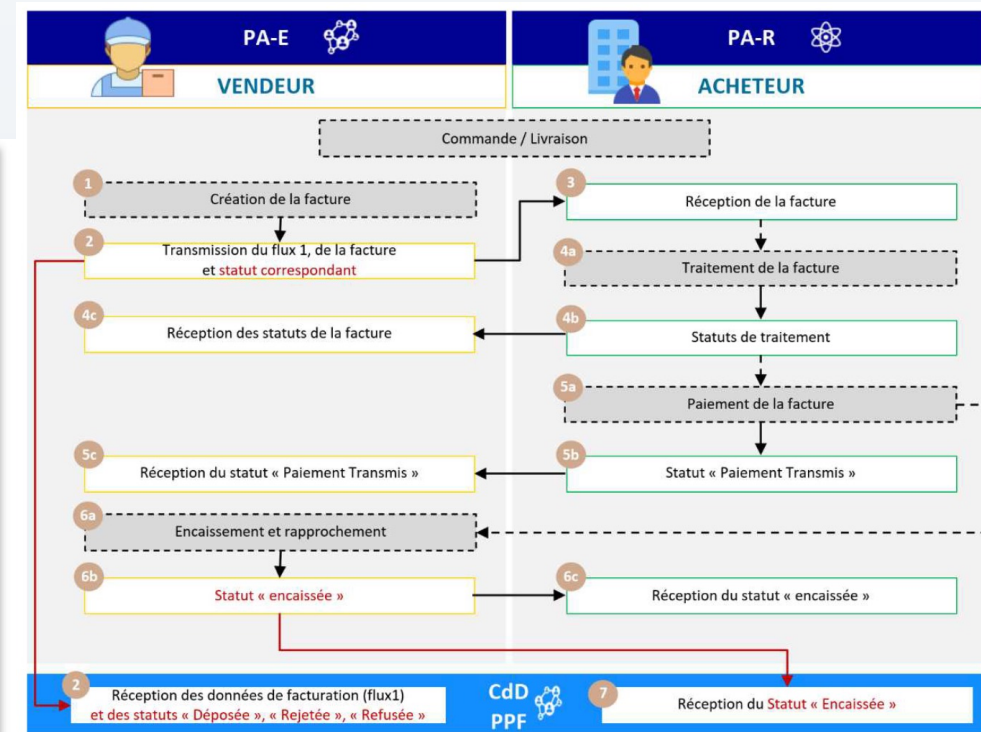


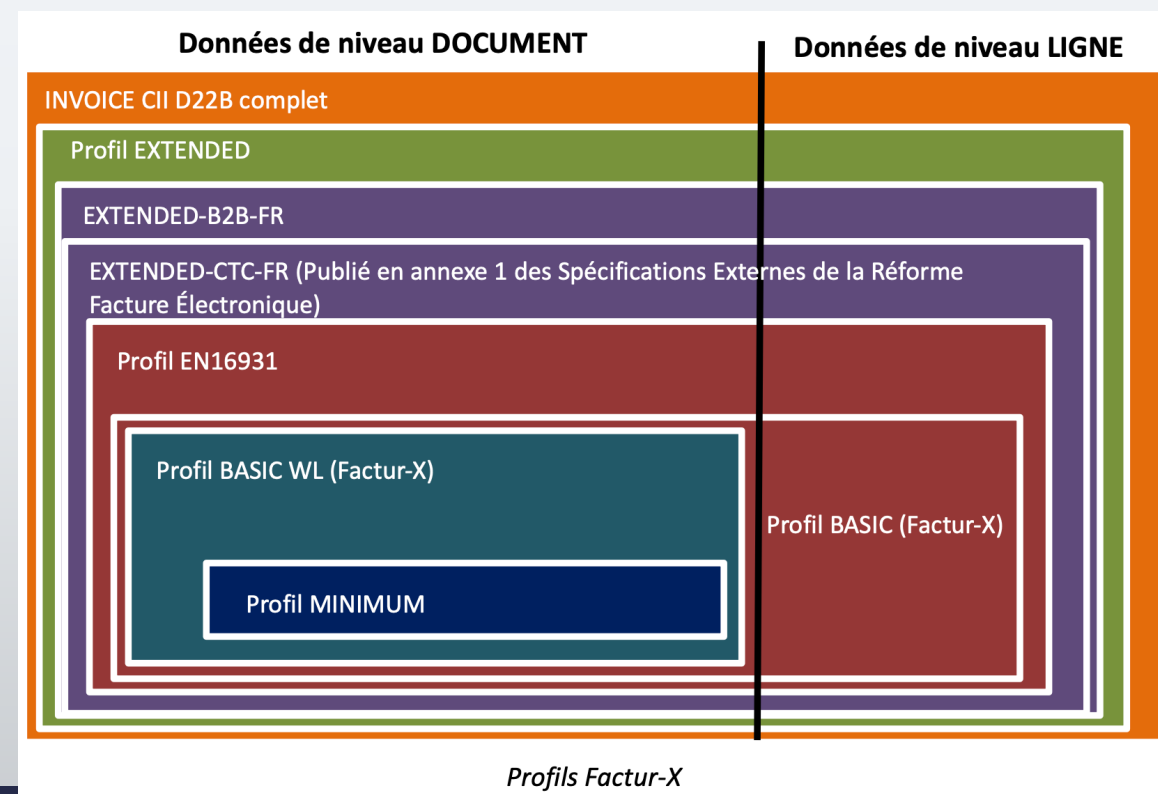
Figure 2 : cas nominal d'échange de facture

Autres cas documentés

- Rejet à l'émission d'une facture e-invoicing
- Facture Déposée NON_TRANSMISE pour absence de PA-R.
- Rejet d'une facture en réception
- Refus d'une facture par l'ACHETEUR (Destinataire de la facture)
- Facture en litige, suivie d'un AVOIR partiel ou total
- Facture en litige, suivie d'une Facture Rectificative

Autres références

- **CDAR D22B (UN/CEFACT)**
 - Cross Domain Acknowledgement and Response
 - Format des accusés de reception entre PDP
- **BRS - CDAError Acknowledgement Process**
 - Processus de gestion des erreurs d'accuse
- **Standard FACTUR-X 1.07.3 2025 05 15 FR VF** disponible sur le site de la fnfe (Forum National de la Facture Électronique et des Marchés Publics Électroniques) (<https://fnfe-mpe.org/>)



[TOUS] Organisation du dépôt

Bloc 4 - Où trouver quoi dans le monorepo

[TOUS] Structure du monorepo

un seul référentiel pour tous les projets



- **./docs/**
 - **briques/** : doc + .feature par brique
 - **développement/** : guides dev, BDD, install
 - **norme/** : normes AFNOR et annexes
 - **test/** : exemples BDD commentés
- **./packages/** : un sous répertoire par « projet »
 - **pac0/** : implémentation de référence
 - **pac-bdd/** : moteur de tests BDD
 - **pac0-cli/** : CLI utilitaire (pilotage de la PA)
- **./docker/**
 - docker-compose (sera déplacé à la racine)
 - Dockerfiles
 - 10 services + test-bdd
- **./report/**
 - Rapports de tests (HTML, MD, XML)
- **./script/**
 - Scripts utilitaires (./script/test)
- **Dépôts :**
 - GitHub : releases publiques
 - Forgejo : développement quotidien

Le « Behavior Driven Development », ou BDD, est une **méthode de développement Agile dans laquelle le produit est conçu autour du comportement qu'un utilisateur s'attend à expérimenter**. Le principe du BDD est donc de préciser un « comportement désiré ».

[TOUS] Stratégie BDD

Bloc 5 - Pourquoi et comment on teste

Le BDD : un langage commun

- **Behavior Driven Development : spécifications exécutables**
 - Langage commun entre métier et développeur
 - Vision communautaire : outils de tests pour les éditeurs de logiciels métier et de PA
- **La boucle 'Trois Amigos'**
 - Expert métier + Développeur + Testeur
- **Gherkin en français**
 - Fonctionnalité, Scenario, Soit, Quand, Alors
- **Cycle de vie d'un test**
 - 1. Rédaction par l'expert métier (.feature)
 - 2. Implémentation par le développeur (step definitions)
 - 3. Exécution automatique (pytest-bdd)

Lien feature -> steps -> systeme

- Fichiers .feature dans `./docs/briques/<NN-nom>/`
 - Rédiges en Gherkin français par les experts métier
- Code des steps dans packages `./pac-bdd/src/pac_bdd/`
 - api.py, peppol.py, service.py, esb.py...
- Le moteur pytest-bdd fait le lien
 - Lit une etape Gherkin (ex: Quand j'appelle l'API GET /healthcheck)
 - Cherche le step definition correspondant
 - Exécute la fonction Python associée
- Rapports : `./report/pac-bdd/`
- Ressources :
 - `./docs/developpement/BDD_README.md`
 - `./docs/developpement/BDD_Guide_Developpeur.md`
 - `./docs/developpement/BDD_Guide_Expert_Metier.md`

[METIER] Rédiger un scénario BDD

Bloc 6 - Guide pour l'expert métier

[METIER] Structure d'un fichier .feature

- **Entête obligatoire : # language: fr**
- **Fonctionnalité: titre unique + description libre**
 - Peut référencer la norme (ex: Section 4.4 de XP_Z12-013.pdf)
- **Contexte: étapes exécutées avant chaque scenario**
 - Soit une pa communautaire
- **Scenario: un cas de test concret**
 - Soit / Etant donne : précondition (Given)
 - Quand : action effectuée (When)
 - Alors : résultat attendu (Then)
 - Et / Mais : continuation
- **Conventions du projet**
 - Variables entre guillemets : "valeur"
 - Constantes préfixées : #pa1, #accepted

Exemple

./docs/briques/01-api-gateway/healthcheck.feature

[METIER] Bonnes pratiques et anti-patterns

- **Déclaratif, pas impératif**
 - Bien : Quand Bob se connecte avec des identifiants valides
 - Eviter : Quand je saisis bob@test.com dans le champ email
- **Un scenario = un comportement (3-5 etapes)**
- **Utiliser des valeurs concrètes**
 - Bien : Alors l'identification par SIRET porte le code "0002"
 - Eviter : Alors l'identification est correcte
- **Anti-patterns à éviter**
 - Détails non essentiels (adresses, CB...)
 - Logique conditionnelle (si... alors...) -> 2 scenarios
- **Fonctionnalités avancées : Plan du Scenario, tableaux, Doc Strings, tags**

[METIER] Exemples concrets du projet

- **Healthcheck simple** ([./docs/briques/01-api-gateway/healthcheck.feature](#))
 - Contexte + 2 scenarios courts, référence norme
- **Calculs PEPPOL** ([./docs/briques/07-routage/peppol.feature](#))
 - Plusieurs scenarios avec valeurs concretes
- **Communication inter-PA** ([./docs/briques/07-routage/pa_multiple.feature](#))
 - References #pa1, #e1, #f1 - test bout en bout
- **Cycle de vie facture** ([./docs/briques/09-gestion-cycle-vie/facture.feature](#))
 - Workflow asynchrone : dépôt puis interrogation
- Exercice : écrire un scenario pour la brique 05, 06 ou 08
- Ressources : [./docs/developpement/BDD_Guide_Expert_Metier.md](#)

[DEV] Installation de l'environnement

Bloc 7 - Mettre en place son poste de développement

[DEV] Accéder au repo

Installer le repo sur ma machine avec git installé

```
mkdir <repertoire racine>  
cd <repertoire racine>  
git clone https://git.pdplibre.org/Construction\_PA/PA\_Communauteaire.git  
cd PA_Communauteaire
```

Accéder au repo via un navigateur :

https://git.pdplibre.org/Construction_PA/PA_Communauteaire.git

[DEV] Installer l'environnement Docker vs Linux natif

- **Avec Docker (recommande)**

- Prerequis : Docker / Podman
- `cd docker && docker compose up -d`
- 10 services + test-bdd en un seul compose
- Verification : `http://localhost:8000/docs`
- Cible : branche `dev_docker_v2`

- **Configuration S3 (stockage factures)**

- Inclus dans le docker-compose (SeaweedFS)
- Port S3 : 8333

`./docs/developpement/Installation_Docker.md`

- **Installation locale Linux**

- Python 3.13+, uv (pas pip !)
- `curl -Lsf https://astral.sh/uv/install.sh | sh`
- uv sync dans chaque package

- **NATS Server (broker de messages)**

- Telecharger et installer nats-server
- Optionnel : nats-cli pour le debug

- **Lancement sequentiel des services**

- 1. `nats-server -V -js`
- 2. `uv run fastapi dev ...` (API Gateway)
- 3. `uv run faststream run ...` (services)

`./docs/developpement/Installation_Linux.md`

[DEV] Verification de l'installation

- **Lancer les tests BDD**

- `cd packages/pac-bdd && uv run pytest -v`

- **Lancer tous les tests avec rapport**

- `./script/test`

- **Verifier l'API Gateway**

- `http://localhost:8000/docs` (interface Swagger FastAPI)

- **Ressources**

- `docs/developpement/Installation_Docker.md`
 - `docs/developpement/Installation_Linux.md`
 - `docs/developpement/Configuration.md`

[DEV] Le code : packages et services

Bloc 8 - Comprendre l'implémentation

[DEV] Implémentation à date (février 2026)

packages/pac0

- **Implémentation de référence**
 - 1 service FastAPI : 01-api-gateway
 - 8 services FastStream : 03 a 09
 - Communication via NATS (JetStream)
- **Structure d'un service**
 - src/pac0/service/<nom>/main.py
- **Fixtures de test partagees**
 - src/pac0/shared/test/
 - WorldContext, PacServiceContext
 - NatsServiceContext

packages/pac-bdd

- **Tests BDD**
 - Point d'entree : test_scenario.py
 - Charge tous les .feature du projet
- **Step definitions**
 - src/pac_bdd/api.py : appels REST
 - src/pac_bdd/peppol.py : routage
 - src/pac_bdd/service.py : services
 - src/pac_bdd/esb.py : bus de messages
 - steps.py : import central

packages/pac0-cli

- uvx pac0-cli@latest run <N>

[DEV] Conventions du code

- **Python 3.13+ avec async/await partout**
- **Package manager : uv (jamais pip)**
 - uv sync pour installer, uv run pour executer
- **pytest-asyncio avec asyncio_mode = "auto"**
- **Licence GPL-3.0-or-later avec headers SPDX**
- **Chaque nouveau module de steps doit etre importe dans steps.py**
- **Ressources**
 - packages/pac0/README.md
 - packages/pac-bdd/README.md

[DEV] Implémenter un test BDD

Bloc 9 - Coder les step definitions

[DEV] Flux : .feature -> step -> systeme

- **1. Identifier l'étape manquante**
 - StepDefNotFound: "je vérifie le format UBL"
- **2. Choisir le bon module**
 - api.py (REST), peppol.py (routage), service.py, esb.py
- **3. Ecrire le step**
 - @when(parsers.parse('j'appelle l'API {verb} {path}'))
 - parsers.parse pour les patterns simples
 - parsers.re pour les regex complexes
- **4. Gérer Data Tables et Doc Strings**
 - datatable : liste de dictionnaires
 - docstring : texte brut du bloc "..."

[DEV] Fixtures et exécution

- **WorldContext : environnement multi-PA**
 - `world1.pa1.api_gateway.get_client()` pour les appels API
- **Contexte local Pydantic (LocalTestCtx)**
 - Partage de données entre steps d'un scenario
- **Pattern async -> sync : `@async_to_sync`**
- **Exécution des tests**
 - `uv run pytest` (tous les tests)
 - `uv run pytest test_scenario.py::test_xxx -v` (un seul)
 - `-v -s --log-cli-level=DEBUG` (mode debug)
 - `--collect-only` (collecter sans executer)
- **Ressources : `docs/developpement/BDD_Guide_Developpeur.md`**

[TOUS] Rapports de tests

Bloc 10 - Vérifier que ça marche

Lancer et lire les rapports

- Lancer tous les tests : `./script/test`
- Lancer un package : `cd packages/pac-bdd && uv run pytest`
- Rapports générés dans `./report/`
 - `report/pac0/` : tests unitaires de l'implémentation
 - `report/pac-bdd/` : tests BDD (scenarios Gherkin)
- 3 formats de rapport
 - HTML : visuel, navigable dans un navigateur
 - Markdown : lisible en texte brut
 - XML JUnit : pour l'intégration continue
- Interpréter un échec
 - `StepDefNotFound` : step manquant (à implémenter)
 - `AssertionError` : bug dans le code ou le scénario

[TOUS] Contribuer

Bloc 11 - Workflow et bonnes pratiques

Cycle de contribution

- **1. Accès Forgejo : demande d'invitation via le forum**
 - <https://forum.pdplibre.org/>
- **2. Fork + clone du projet depuis Forgejo**
 - `git clone https://git.pdplibre.org/Construction_PA/PA_Communauteaire.git`
- **3. Creer une branche thématique**
 - `feature/ma-fonctionnalite`, `fix/mon-correctif`
- **4. Verifier les tests avant de commiter**
 - `cd packages/pac-bdd && uv run pytest -v`
- **5. Ouvrir une Pull Request**
 - Toujours en lien avec une Issue
 - Prefixe WIP tant que le travail est en cours
- **6. Licence GPL-3.0-or-later, headers SPDX obligatoires**

[METIER] Focus par brique : écrire les tests

Bloc 12 - Couvrir le périmètre fonctionnel

- **Briques avec .feature existants mais incomplet**

- 01-API Gateway : healthcheck, service, sha256, trackingId
- 02-ESB Central : healthcheck, esb, service, lifecycle, world
- 03-Controle Formats : format, service
- 04-Validation Metier : compliance
- 07-Routage : peppol, pa_multiple, peppol_live
- 09-Gestion Cycle Vie : facture, workflow, demo

- **Briques a couvrir (prioritaires)**

- 05-Conversion Formats
- 06-Annuaire Local
- 08-Transmission Fiscale
- 10-Stockage

- **Méthode**

- 1. Lire le README de la brique
- 2. Identifier la section de norme
- 3. Lister les comportements attendus
- 4. Rediger les scenarios .feature

Synthese

Recap des parcours et ressources

Recapitulatif des parcours

Bloc	DEV	METIER	Theme
1 - Contexte et vision	X	X	Pourquoi ce projet existe
2 - Architecture 10 briques	X	X	Organisation du système
3 - Normes de référence	X	X	Sur quoi on s'appuie
4 - Organisation du dépôt	X	X	Où trouver quoi
5 - Stratégie BDD	X	X	Comment on teste
6 - Rédiger un scénario BDD		X	Ecrire un .feature
7 - Installation environnement	X		Mettre en place son poste
8 - Le code : packages	X		Comprendre l'implémentation
9 - Implémenter un test BDD	X		Coder les steps
10 - Rapports de tests	X	X	Vérifier que ça marche
11 - Contribuer	X	X	Pousser ses changements
12 - Focus par brique		X	Couvrir le périmètre

Ressources et contacts

- **Documentation**

- docs/developpement/Architecture.md - Architecture technique
- docs/developpement/Contribuer.md - Guide de contribution
- docs/developpement/BDD_Guide_Expert_Metier.md - Guide BDD metier
- docs/developpement/BDD_Guide_Developpeur.md - Guide BDD dev
- docs/norme/index.md - Normes de reference

- **Communauté**

- Forum : <https://forum.pdplibre.org/>
- Forgejo : https://git.pdplibre.org/Construction_PA/PA_Communaute
- GitHub : https://github.com/PDP-Libre/PA_Communaute
- Visio bimensuelle : [C'est par ici](#)



Merci

pdlibre.org

contact@pdlibre.org